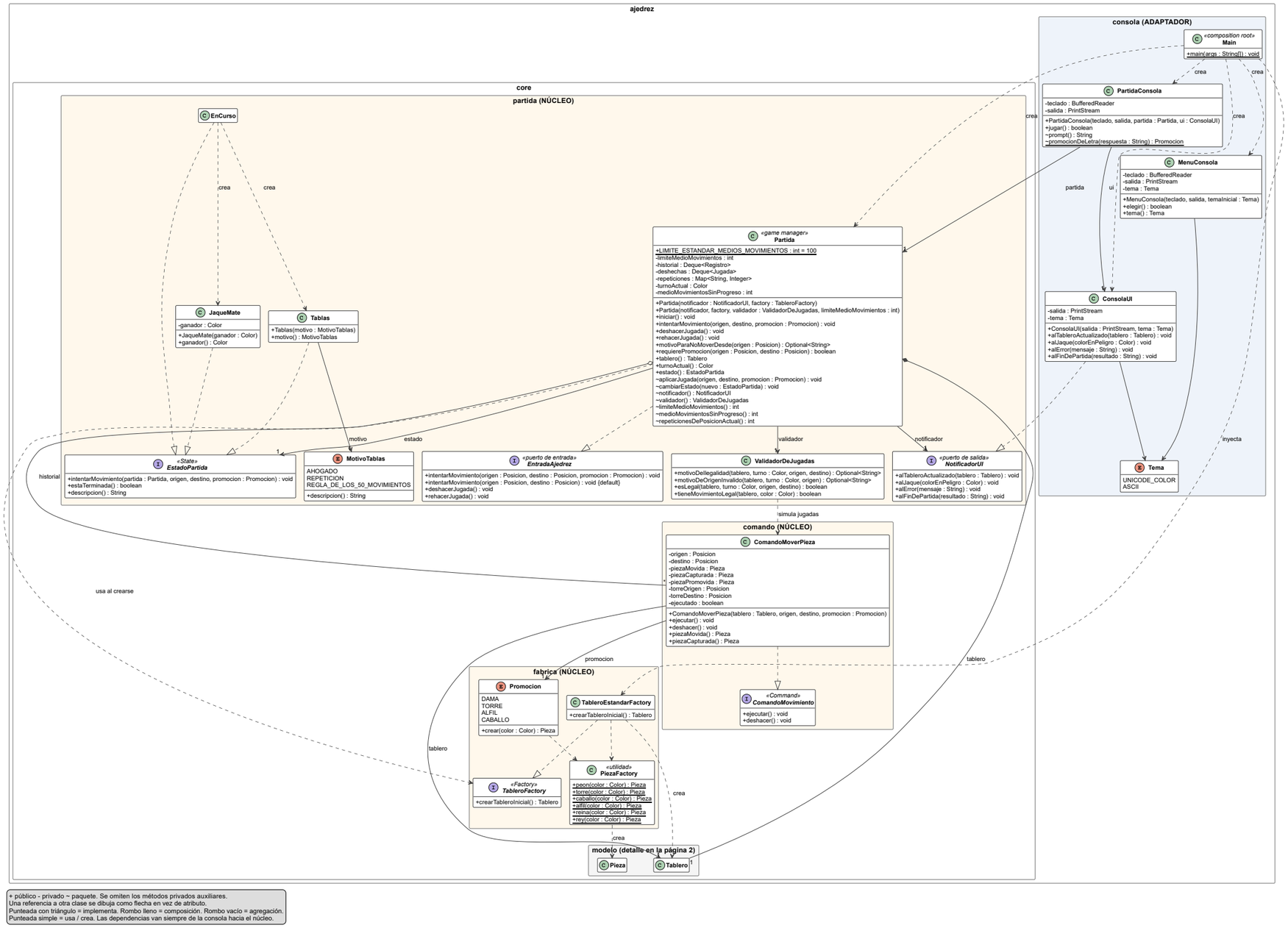
1. Arquitectura: consola (adaptador), partida, comandos y fábricas



+ público - privado ~ paquete. Se omiten los métodos privados auxiliares.
Una referencia a otra clase se dibuja como flecha en vez de atributo.
Punteada con triángulo = implementa. Rombo lleno = composición. Rombo vacío = agregación.
Punteada simple = usa / crea. Las dependencias van siempre de la consola hacia el núcleo.

Puerto de entrada (EntradaAjedrez): es la vía por donde la interfaz le pide a la Partida que intente mover una pieza (indicando, si corresponde, en qué pieza promociona un peón), deshaga o rehaga una jugada.

Puerto de salida (NotificadorUI): es la abstracción que usa la Partida para avisarle al exterior que hay que redibujar el tablero, mostrar un jaque, un error o el final de la partida. Como la Partida depende de esta interfaz y no de una clase concreta, el mismo núcleo se puede conectar a la consola (ConsolaUI) o a una app móvil sin tocar una sola línea de la lógica interna.

Partida: funciona como el cerebro central ("game manager"). Mantiene las referencias clave: el Tablero, el notificador, el validador, de quién es el turno (un Color que se alterna), la fase de la partida y el historial de jugadas para deshacer y rehacer. No sabe que existe una consola.

Fases de la partida (State): EnCurso procesa las jugadas; JaqueMate y Tablas las rechazan. Agregar una fase nueva es agregar una clase, sin tocar los if de las demás.

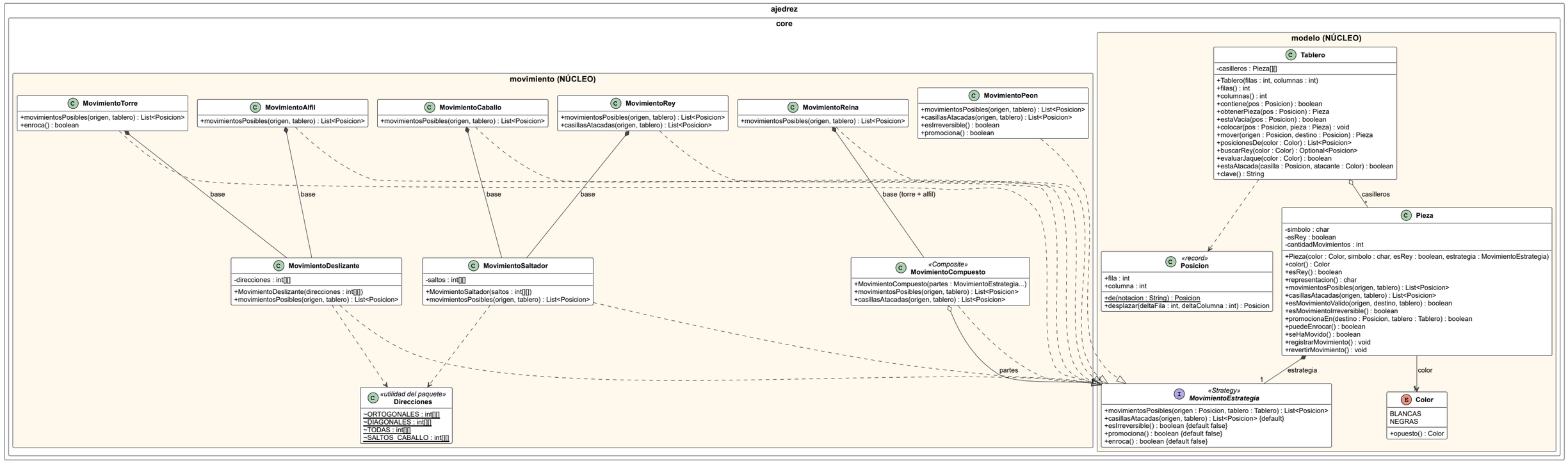
ValidadorDeJugadas: decide si una jugada es legal: que respete el movimiento de la pieza y que no deje al propio rey en jaque. Para eso prueba la jugada con un comando y la deshace.

ComandoMoverPieza (Command): encapsula una jugada y guarda lo necesario para revertirla exactamente: la pieza capturada, el peón que promocionó y la torre que saltó en un enroque.

Fábricas: la TableroFactory se inyecta en la Partida, así que jugar con otro tablero o con otra posición inicial no modifica el núcleo. Promocion crea la pieza elegida al coronar.

Consola (adaptador): Main arma todo y lo inyecta; PartidaConsola lee lo que escribe el jugador; ConsolaUI dibuja; MenuConsola permite elegir el tema. Todas las dependencias apuntan hacia el núcleo.

2. Modelo y estrategias de movimiento (Strategy + Composite)



Posicion: al ser una clase independiente (un record) encapsula la fila y la columna, permitiendo que todas las demás clases se pasen coordenadas de forma limpia y estructurada mediante relaciones de uso (líneas punteadas). Posicion.de("e2") traduce la notación del jugador.

Tablero: maneja la matriz de casilleros. Su tamaño (filas y columnas) se recibe en el constructor, no está fijo en 8x8. Sabe si una casilla está atacada y si un rey está en jaque.

Pieza: tiene un Color y una MovimientoEstrategia. No existe una subclase por tipo de pieza: lo que distingue a una torre de un caballo es la estrategia que recibe (composición sobre herencia).

MovimientoEstrategia (Strategy): define cómo se mueve una pieza y qué casillas amenaza. Tres propiedades tienen un valor por defecto (si es irreversible, si promociona, si enroca), así una pieza nueva no necesita implementarlas.

Bloques reutilizables: torre y alfil se deslizan (MovimientoDeslizante); caballo y rey saltan (MovimientoSaltador); la reina es un MovimientoCompuesto de torre + alfil (Composite).

Extensibilidad: para agregar una pieza nueva alcanza con implementar MovimientoEstrategia o combinar las existentes, y crearla con su símbolo. Ninguna clase existente se modifica.

Fuente editable: uml/2-modelo-y-movimientos.puml (PlantUML).